- Composite (builds a tree structure of related objects that behaves as a single object)
- Facade (wraps a complex interface in a simplified version)
- Flyweight (minimize memory use by allowing objects to share common data)
- Proxy (provides a substitute object, allowing additional functionality before or after calling the target)

Behavioral Patterns

We've seen *how* to build objects, we've considered *what* objects to build, and now we'll look at patterns for controlling how our objects behave as they do their work. Behavioral patterns provide a set of templates that are primarily concerned with how objects communicate with each other and also how they can alter their behavior as your program executes.

One idea that repeatedly crops up in OOP (and to be fair, programming in general) is the idea of *coupling*. Coupling is the degree to which one component is aware of (and thus dependent on) another. In particular, *loose coupling* is the ideal: where components are weakly associated (so that changes in one will have the most reduced effect on another), and there is little to no special knowledge of one component in another. Components that are loosely coupled are easily altered or replaced, which means maintaining code is less expensive (both in monetary cost and engineer well-being). The behavioral patterns are all designed with this goal in mind, so let's look at an example.

The problem case we'll consider is one where a range of possible behaviors (or algorithms) exists, only one of which is required and can only be determined at runtime. This is a very common problem, and the naive solution is to wrap each code block that defines the behavior in a construct such as a `switch` block or a set of `if` statements (we've seen these in Chapter 4, "Logic and Control"). To give an example scenario: you might have an application that serves up data to a web client in a variety of file formats, maybe as delimited values (such as CSV, PSV, etc.) or a full-blown spreadsheet. The application retrieves data from a database and converts it to the relevant format based on information contained in the HTTP request. The algorithm (build a comma-separated file, pipe-separated, tab-separated, or build something in a spreadsheet